

PORT SCANNER

Complete Command Reference

Intelligent Port Scanner & Security Risk Analyzer
Built with Python Socket Library · No External Scanner Used

Version: Port Scanner v2.1
Platform: Linux CLI (Ubuntu / Kali)
Language: Python 3.10+
Modules: 8 Python modules · ~1200 lines
Entry Point: python3 portsentinel.py
Generated: 30 June 2026

LEGAL DISCLAIMER

Only scan systems you own or have explicit written permission to test. Unauthorized port scanning may be illegal in your jurisdiction.

Section 1 — Setup & Installation

Step 1 — Extract and enter the project

```
# Extract the zip archive
unzip PortScanner_v2_1.zip
cd PortScanner

# Verify folder structure
ls -la
```

Step 2 — Install optional dependencies

```
# SSH default credential checking (recommended)
pip install paramiko --break-system-packages

# SSH cipher audit tool (optional, deeper audit)
pip install ssh-audit --break-system-packages

# Or install via apt
sudo apt install ssh-audit -y
```

Step 3 — Make executable (optional convenience)

```
chmod +x portsentinel.py

# Now you can run as:
./portsentinel.py -t 127.0.0.1

# Or always with python3:
python3 portsentinel.py -t 127.0.0.1
```

Step 4 — Verify installation

```
# Show full help menu
python3 portsentinel.py --help

# Quick test against localhost
python3 portsentinel.py -t 127.0.0.1 -p 1-1024
```

Section 2 — Basic Scanning

Quick Scan Reference

Command	Description
python3 portsentinel.py -t 192.168.1.1	Default scan — TCP ports 1-1024
python3 portsentinel.py -t 127.0.0.1	Scan localhost
python3 portsentinel.py -t example.com	Scan by hostname
python3 portsentinel.py -t 192.168.1.1 -p 22,80,443	Scan specific ports
python3 portsentinel.py -t 192.168.1.1 -p 1-1024	Scan port range 1 to 1024
python3 portsentinel.py -t 192.168.1.1 -p 1-65535	Scan ALL 65,535 ports
python3 portsentinel.py -t 192.168.1.1 -p 1-1024 -v	Verbose — show closed ports too

Target Examples

```
# Local machine
python3 portsentinel.py -t 127.0.0.1

# Local network host
python3 portsentinel.py -t 192.168.1.100

# By hostname
python3 portsentinel.py -t scanme.nmap.org -p 1-1024

# Common web server ports
python3 portsentinel.py -t 192.168.1.1 -p 80,443,8080,8443,8000,8888

# Common database ports
python3 portsentinel.py -t 192.168.1.1 -p 3306,5432,1433,6379,27017,5984

# Common remote access ports
python3 portsentinel.py -t 192.168.1.1 -p 22,23,3389,5900,5901
```

Section 3 — Scan Types & Performance

Scan Type Selection

	Description
--scan-type tcp	TCP scan only (default) — full connect scan
--scan-type udp	UDP scan — connectionless, stateless
--scan-type both	Run TCP + UDP scan on same target
--threads 100	Number of concurrent threads (default: 100)
--threads 200	High-speed scan — use for large ranges
--threads 20	Low-speed scan — avoid triggering IDS
--timeout 1.0	Connection timeout in seconds (default: 1.0)
--timeout 0.5	Faster — may miss filtered ports
--timeout 3.0	Slower — better accuracy on slow networks

Performance Examples

```
# Default speed (100 threads, 1.0s timeout)
python3 portsentinel.py -t 192.168.1.1 -p 1-1024

# Maximum speed — large range
python3 portsentinel.py -t 192.168.1.1 -p 1-65535 --threads 200 --timeout 0.5

# Stealthy — low profile (avoid IDS/fail2ban)
python3 portsentinel.py -t 192.168.1.1 -p 1-1024 --threads 10 --timeout 3.0

# UDP scan — common UDP services
python3 portsentinel.py -t 192.168.1.1 --scan-type udp -p 53,67,68,123,161,500

# Both TCP and UDP
python3 portsentinel.py -t 192.168.1.1 --scan-type both -p 1-1024
```

Expected Scan Duration

Ports	Threads	Timeout	Est. Duration	Use Case
1,024	100	1.0s	~10-15 sec	Standard audit
1,024	200	0.5s	~3-5 sec	Quick check
65,535	100	1.0s	~11-18 min	Full discovery
65,535	200	0.5s	~5-9 min	Fast full scan
20 ports	100	1.0s	<2 sec	Targeted check

Section 4 — Feature Flags

Individual Feature Commands

	Description
--banner	Enable service banner grabbing — detect versions
--risk	Enable risk scoring — CRITICAL/HIGH/MEDIUM/LOW
--os-detect	Enable OS fingerprinting via TTL analysis
--exploit	Run full exploitation awareness checks
--no-creds	Skip credential brute-force (exploit mode)
--no-ssh-audit	Skip SSH cipher audit (exploit mode)
--full-scan	Enable all: banner + risk + os-detect + exploit
--verbose / -v	Show closed and filtered ports in output
--no-color	Disable ANSI colors (for logging/piping)

Feature Combination Examples

```
# Banner grabbing only
python3 portsentinel.py -t 192.168.1.1 --banner

# Risk scoring only
python3 portsentinel.py -t 192.168.1.1 --risk

# Banner + risk together
python3 portsentinel.py -t 192.168.1.1 --banner --risk

# OS fingerprinting only
python3 portsentinel.py -t 192.168.1.1 --os-detect

# Exploitation awareness — all checks
python3 portsentinel.py -t 192.168.1.1 -p 21,22,23 --exploit

# Exploit check — SSH ciphers + anon FTP only (no cred brute)
python3 portsentinel.py -t 192.168.1.1 --exploit --no-creds

# Exploit check — credentials only (no SSH cipher audit)
python3 portsentinel.py -t 192.168.1.1 --exploit --no-ssh-audit

# Full scan — everything enabled
python3 portsentinel.py -t 192.168.1.1 --full-scan

# Full scan with verbose output
python3 portsentinel.py -t 192.168.1.1 --full-scan --verbose
```

Section 5 — Report Generation

Report Format Options

	Description
<code>--report html</code>	Generate HTML dashboard report
<code>--report txt</code>	Generate plain text log
<code>--report json</code>	Generate structured JSON data
<code>--report all</code>	Generate all 3 formats simultaneously
<code>--output-dir ./reports</code>	Save reports to custom directory (default: reports/)

Report Commands

```
# HTML report (browser viewable)
python3 portsentinel.py -t 192.168.1.1 --full-scan --report html

# JSON report (machine readable)
python3 portsentinel.py -t 192.168.1.1 --full-scan --report json

# Plain text report
python3 portsentinel.py -t 192.168.1.1 --full-scan --report txt

# All 3 formats at once
python3 portsentinel.py -t 192.168.1.1 --full-scan --report all

# Save to custom folder
python3 portsentinel.py -t 192.168.1.1 --full-scan --report all --output-dir ./my_reports

# View generated report files
ls -lh reports/

# Open HTML report in browser
xdg-open reports/scan_*.html
```

Output & Logging Commands

```
# Save terminal output to file
python3 portsentinel.py -t 192.168.1.1 --full-scan | tee scan_output.txt

# Save with no color (clean plain log)
python3 portsentinel.py -t 192.168.1.1 --full-scan --no-color | tee scan_log.txt

# Save with timestamp in filename
python3 portsentinel.py -t 192.168.1.1 --full-scan --no-color \
| tee scan_$(date +%Y%m%d_%H%M%S).txt

# Grep only open ports from output
python3 portsentinel.py -t 192.168.1.1 --no-color | grep OPEN

# Count total open ports
python3 portsentinel.py -t 192.168.1.1 --no-color | grep -c OPEN
```

Section 6 — Real-World Scan Combinations

Quick recon on a new target

```
python3 portsentinel.py -t 192.168.1.1 -p 1-1024 --banner --risk
```

Standard security audit (common ports)

```
python3 portsentinel.py -t 192.168.1.1 -p 1-1024 --full-scan --report html
```

Deep full audit (all ports)

```
python3 portsentinel.py -t 192.168.1.1 -p 1-65535 --full-scan --threads 150 --report all
```

Web server audit only

```
python3 portsentinel.py -t 192.168.1.1 -p 80,443,8080,8443,8000 --banner --risk
```

Database exposure check

```
python3 portsentinel.py -t 192.168.1.1 -p 3306,5432,1433,6379,27017 --banner --risk
```

Remote access audit

```
python3 portsentinel.py -t 192.168.1.1 -p 22,23,3389,5900 --exploit
```

Exploitation check (FTP + SSH + Telnet)

```
python3 portsentinel.py -t 192.168.1.1 -p 21,22,23 --exploit --report all
```

SSH security audit only

```
python3 portsentinel.py -t 192.168.1.1 -p 22 --exploit --no-creds
```

FTP anonymous check only

```
python3 portsentinel.py -t 192.168.1.1 -p 21 --exploit --no-creds --no-ssh-audit
```

Stealthy low-noise scan

```
python3 portsentinel.py -t 192.168.1.1 -p 1-1024 --threads 10 --timeout 3.0
```

Fast high-speed full discovery

```
python3 portsentinel.py -t 192.168.1.1 -p 1-65535 --threads 200 --timeout 0.5
```

Save everything for report

```
python3 portsentinel.py -t 192.168.1.1 --full-scan --report all --no-color | tee audit.txt
```

Section 7 — Port Risk Quick Reference

Common ports and their risk levels identified by the risk engine:

Port	Service	Protocol	Risk	Why Dangerous
21	FTP	TCP	HIGH	Cleartext credentials. Often allows anonymous login.
22	SSH	TCP	MEDIUM	Encrypted but target of brute-force. Weak ciphers possible.
23	Telnet	TCP	CRITICAL	All data sent in plaintext — passwords visible on network.
25	SMTP	TCP	MEDIUM	Mail relay abuse, open relay misconfiguration.
53	DNS	UDP	MEDIUM	DNS amplification attacks, zone transfer leaks.
80	HTTP	TCP	LOW	Unencrypted web — sniffable. Redirect to HTTPS.
110	POP3	TCP	HIGH	Cleartext email credentials. Use POP3S (995).
143	IMAP	TCP	HIGH	Cleartext email access. Use IMAPS (993).
443	HTTPS	TCP	LOW	Encrypted. Check cert validity and TLS version.
445	SMB	TCP	CRITICAL	EternalBlue (MS17-010), ransomware entry point.
3306	MySQL	TCP	HIGH	DB exposed to network. Bind to localhost only.
3389	RDP	TCP	HIGH	BlueKeep CVE-2019-0708. Brute-force target.
5432	PostgreSQL	TCP	HIGH	Database exposed. Should be localhost only.
6379	Redis	TCP	CRITICAL	Often runs without auth. Allows arbitrary code exec.
27017	MongoDB	TCP	CRITICAL	Default: no auth. Exposed DBs wiped by attackers.

Scan by Risk Category

```
# Scan CRITICAL risk ports only
python3 portsentinel.py -t 192.168.1.1 -p 23,445,6379,27017 --risk --banner

# Scan HIGH risk ports
python3 portsentinel.py -t 192.168.1.1 -p 21,110,143,3306,3389,5432 --risk --banner

# Check all database ports
python3 portsentinel.py -t 192.168.1.1 -p 3306,5432,1433,1521,6379,27017,5984,9200 --risk

# Check all remote access ports
python3 portsentinel.py -t 192.168.1.1 -p 22,23,3389,5900,5901,5902 --exploit
```


Section 8 — Exploitation Awareness Commands

Anonymous FTP Testing

```
# Check if FTP allows anonymous login
python3 portsentinel.py -t 192.168.1.1 -p 21 --exploit --no-creds --no-ssh-audit

# FTP full check (anon login + default credentials)
python3 portsentinel.py -t 192.168.1.1 -p 21 --exploit --no-ssh-audit

# FTP on non-standard port
python3 portsentinel.py -t 192.168.1.1 -p 2121 --exploit --no-creds
```

SSH Security Audit

```
# SSH cipher audit only (no credential testing)
python3 portsentinel.py -t 192.168.1.1 -p 22 --exploit --no-creds

# SSH default credential check (requires paramiko)
pip install paramiko --break-system-packages
python3 portsentinel.py -t 192.168.1.1 -p 22 --exploit --no-ssh-audit

# SSH on alternate port
python3 portsentinel.py -t 192.168.1.1 -p 2222 --exploit

# Full SSH audit (ciphers + credentials)
python3 portsentinel.py -t 192.168.1.1 -p 22 --exploit
```

Telnet Security Check

```
# Check if Telnet is open (already critical)
python3 portsentinel.py -t 192.168.1.1 -p 23 --risk

# Telnet default credential testing
python3 portsentinel.py -t 192.168.1.1 -p 23 --exploit

# Full exploit check — FTP + SSH + Telnet
python3 portsentinel.py -t 192.168.1.1 -p 21,22,23 --exploit --report html
```

Exploitation Awareness Flags

	Description
--exploit	Run all exploitation awareness checks
--no-creds	Skip default credential brute-force
--no-ssh-audit	Skip SSH cipher algorithm audit
--timeout 5.0	Increase timeout for slow auth responses

IMPORTANT — Rate Limiting

SSH servers with fail2ban enabled will reset connections after repeated attempts. The tool auto-stops after 3 consecutive resets and adds 0.5s delay between each attempt. This is normal behavior — it means the server has brute-force protection active.

Section 9 — Complete Flag Reference

	Default	Description
-t / --target	Required	Target IP address or hostname to scan
-p / --ports	1-1024	Port range (1-1024) or list (22,80,443)
--scan-type	tcp	Scan type: tcp udp both
--threads	100	Concurrent threads for scanning
--timeout	1.0	Connection timeout in seconds
--full-scan	off	Enable all features: banner+risk+os+exploit
--banner	off	Enable service banner grabbing
--risk	off	Enable risk scoring engine
--os-detect	off	Enable OS fingerprinting via TTL
--exploit	off	Run exploitation awareness checks
--no-creds	off	Skip credential brute-force in exploit mode
--no-ssh-audit	off	Skip SSH cipher audit in exploit mode
--verbose / -v	off	Show closed and filtered ports in output
--report	None	Report format: html txt json all
--output-dir	reports/	Directory to save generated report files
--no-color	off	Disable ANSI color codes in terminal output
--help / -h	—	Show full help menu and exit

Syntax Pattern

```
python3 portsentinel.py -t [options]

# Full syntax example
python3 portsentinel.py \
  -t 192.168.1.1 \
  -p 1-1024 \
  --scan-type tcp \
  --threads 100 \
  --timeout 1.0 \
  --full-scan \
  --report all \
  --output-dir ./reports
```

Section 10 — One-Page Cheatsheet

Setup

```
unzip PortScanner_v2_1.zip && cd PortScanner
pip install paramiko --break-system-packages
python3 portsentinel.py --help
```

Basic Scan

```
python3 portsentinel.py -t 192.168.1.1
python3 portsentinel.py -t 192.168.1.1 -p 22,80,443
python3 portsentinel.py -t 192.168.1.1 -p 1-65535
```

TCP / UDP

```
python3 portsentinel.py -t 192.168.1.1 --scan-type tcp
python3 portsentinel.py -t 192.168.1.1 --scan-type udp -p 53,161
python3 portsentinel.py -t 192.168.1.1 --scan-type both
```

Features

```
python3 portsentinel.py -t 192.168.1.1 --banner
python3 portsentinel.py -t 192.168.1.1 --risk
python3 portsentinel.py -t 192.168.1.1 --os-detect
python3 portsentinel.py -t 192.168.1.1 --exploit
```

Full Scan

```
python3 portsentinel.py -t 192.168.1.1 --full-scan
python3 portsentinel.py -t 192.168.1.1 --full-scan --verbose
python3 portsentinel.py -t 192.168.1.1 --full-scan --report all
```

Reports

```
python3 portsentinel.py -t 192.168.1.1 --full-scan --report html
python3 portsentinel.py -t 192.168.1.1 --full-scan --report json
python3 portsentinel.py -t 192.168.1.1 --full-scan --report all
```

Logging

```
python3 portsentinel.py -t 192.168.1.1 --full-scan | tee out.txt
python3 portsentinel.py -t 192.168.1.1 --full-scan --no-color | tee log.txt
python3 portsentinel.py -t 192.168.1.1 --no-color | grep OPEN
```

Speed Control

```
python3 portsentinel.py -t 192.168.1.1 --threads 200 --timeout 0.5
python3 portsentinel.py -t 192.168.1.1 --threads 10 --timeout 3.0
```

TIP

Always scan only systems you own or have explicit written permission to test. For the help menu at any time: *python3 portsentinel.py --help*